K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Lab Assignment-3

School of Engineering and Technology

Name = Tamanna

RollNo. = 2401201138

Program =BCA (AI & DS)”A

Fundamental of Operating System

Lab ENCA252

Submitted to :- Jyoti Yadav

Comprehensive Implementation of Page Replacement Algorithms

Problem Statement

In modern operating systems, virtual memory allows efficient utilization of physical memory. When memory frames are full, page replacement algorithms are used to decide which page should be replaced.

This assignment focuses on implementing and analyzing **all major page replacement algorithms,** including **FIFO, LRU, Optimal, MRU, and Second Chance (Clock Algorithm).** Students will simulate memory behavior and evaluate performance based on page faults.

Tools/Technology Used

- Python 3.x
- Basic Python libraries
- Ubuntu

Learning Objectives

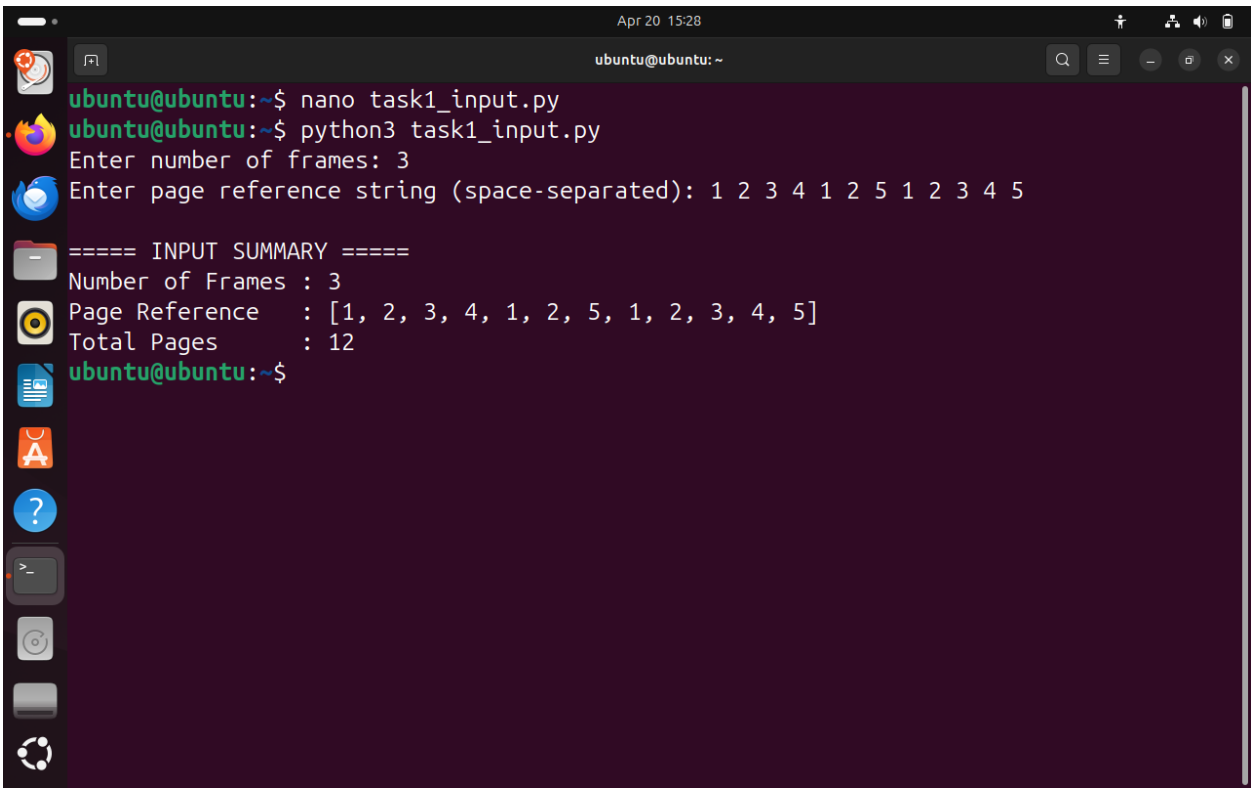
- Understand virtual memory and paging
- Implement multiple page replacement algorithms
- Calculate page faults
- Compare efficiency of algorithms

Task 1: Input and Page Reference String

Code: -

```
num_frames = int(input("Enter number of frames: "))
pages = list(map(int, input("Enter page reference string: ").split()))
print(f"\nFrames : {num_frames}")
print(f"Pages  : {pages}")
print(f"Total  : {len(pages)}")
```

Output:

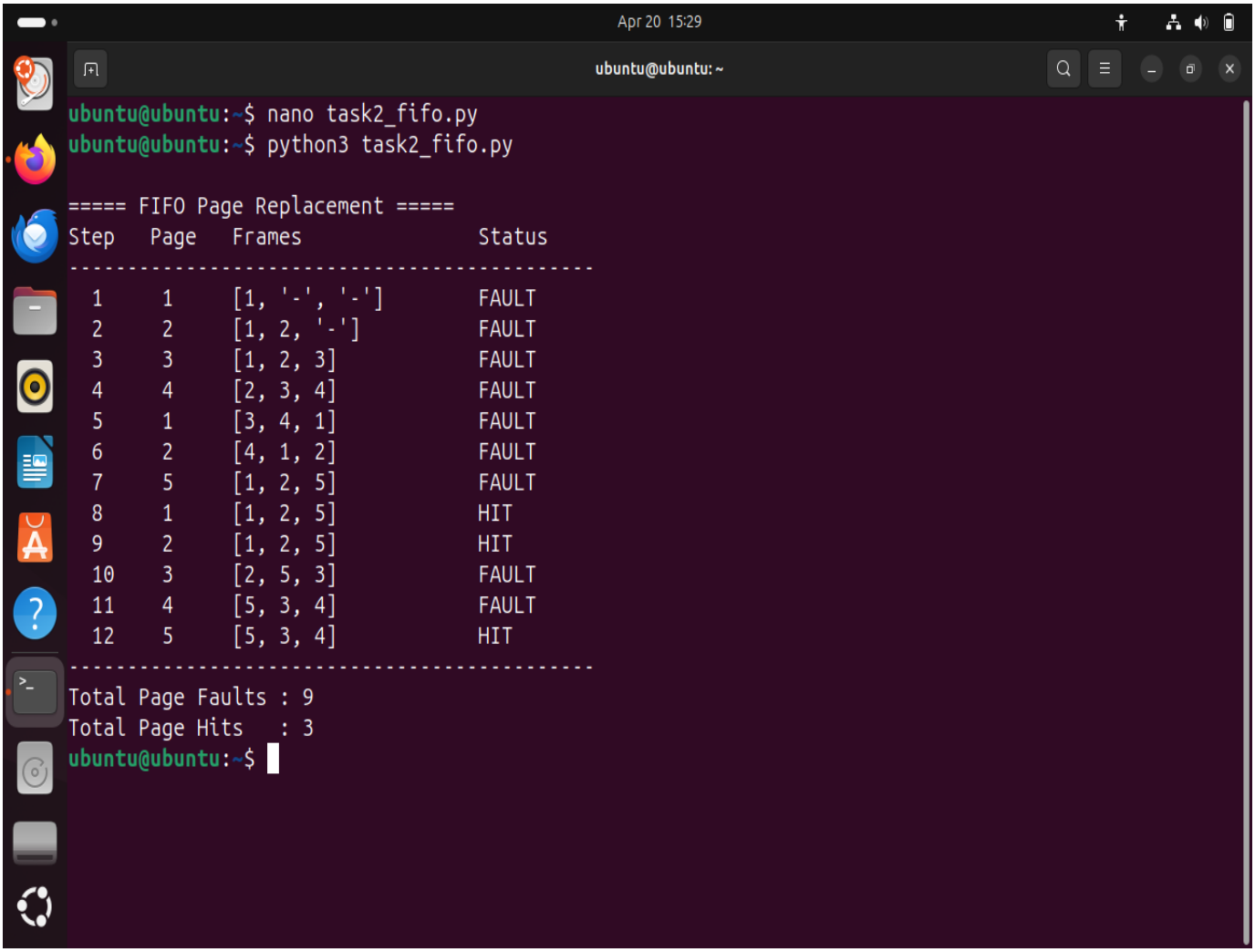


Task 2: FIFO (First-In-First-Out)

Code: -

```
from collections import deque
pages = list(map(int, input("Enter pages: ").split()))
num_frames = int(input("Enter frames: "))
frames, frame_set, faults = deque(), set(), 0
print("\n===== FIFO =====")
for i, page in enumerate(pages):
    fault = False
    if page not in frame_set:
        fault = True
        faults += 1
        if len(frames) < num_frames:
            frames.append(page)
        else:
            removed = frames.popleft()
            frame_set.discard(removed)
            frames.append(page)
        frame_set.add(page)
    cur = list(frames) + ['-'] * (num_frames - len(frames))
    print(f"Step {i+1} | Page {page} | {cur} | {'FAULT' if fault else 'HIT'}")
print(f"Total Faults: {faults}")
```

Output: -



```
ubuntu@ubuntu:~$ nano task2_fifo.py
ubuntu@ubuntu:~$ python3 task2_fifo.py

===== FIFO Page Replacement =====
Step   Page   Frames              Status
---------------------------------------------
  1     1     [1, '-', '-']       FAULT
  2     2     [1, 2, '-']         FAULT
  3     3     [1, 2, 3]           FAULT
  4     4     [2, 3, 4]           FAULT
  5     1     [3, 4, 1]           FAULT
  6     2     [4, 1, 2]           FAULT
  7     5     [1, 2, 5]           FAULT
  8     1     [1, 2, 5]           HIT
  9     2     [1, 2, 5]           HIT
  10    3     [2, 5, 3]           FAULT
  11    4     [5, 3, 4]           FAULT
  12    5     [5, 3, 4]           HIT
---------------------------------------------
Total Page Faults : 9
Total Page Hits   : 3
ubuntu@ubuntu:~$
```

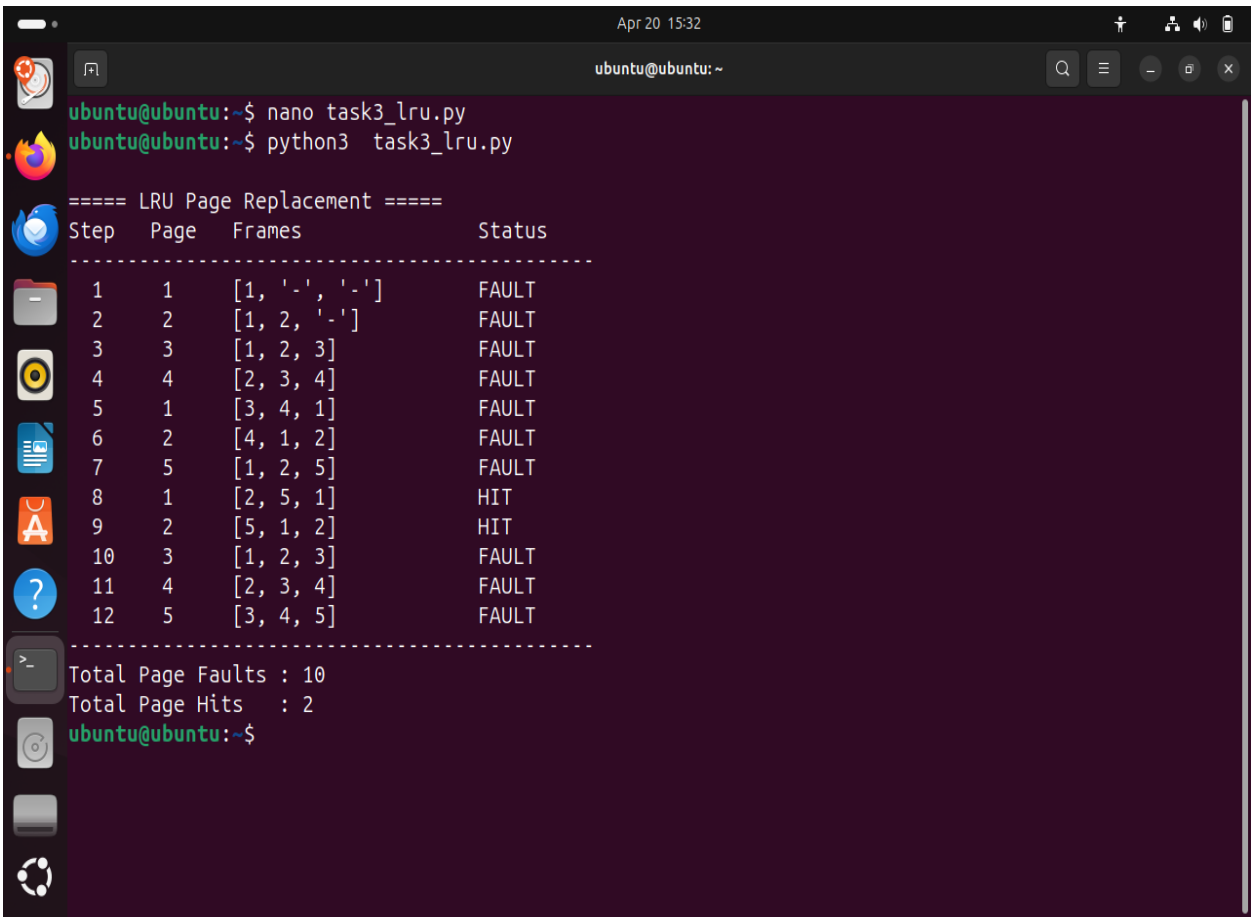
Task 3: LRU (Least Recently Used)

Code : -

```
pages = list(map(int, input("Enter pages: ").split()))
num_frames = int(input("Enter frames: "))

frames, faults = [], 0
print("\n===== LRU =====")
for i, page in enumerate(pages):
    fault = False
    if page not in frames:
        fault = True
        faults += 1
        if len(frames) < num_frames:
            frames.append(page)
        else:
            lru_page, lru_time = None, float('inf')
            for f in frames:
                last = next((j for j in range(i-1,-1,-1) if pages[j]==f), -1)
                if last < lru_time:
                    lru_time, lru_page = last, f
            frames.remove(lru_page)
            frames.append(page)
    else:
        frames.remove(page)
        frames.append(page)
```

```
    cur = frames[:] + ['-'] * (num_frames - len(frames))
    print(f"Step {i+1} | Page {page} | {cur} | {'FAULT' if fault else 'HIT'}")
print(f"Total Faults: {faults}")
```

Output: -

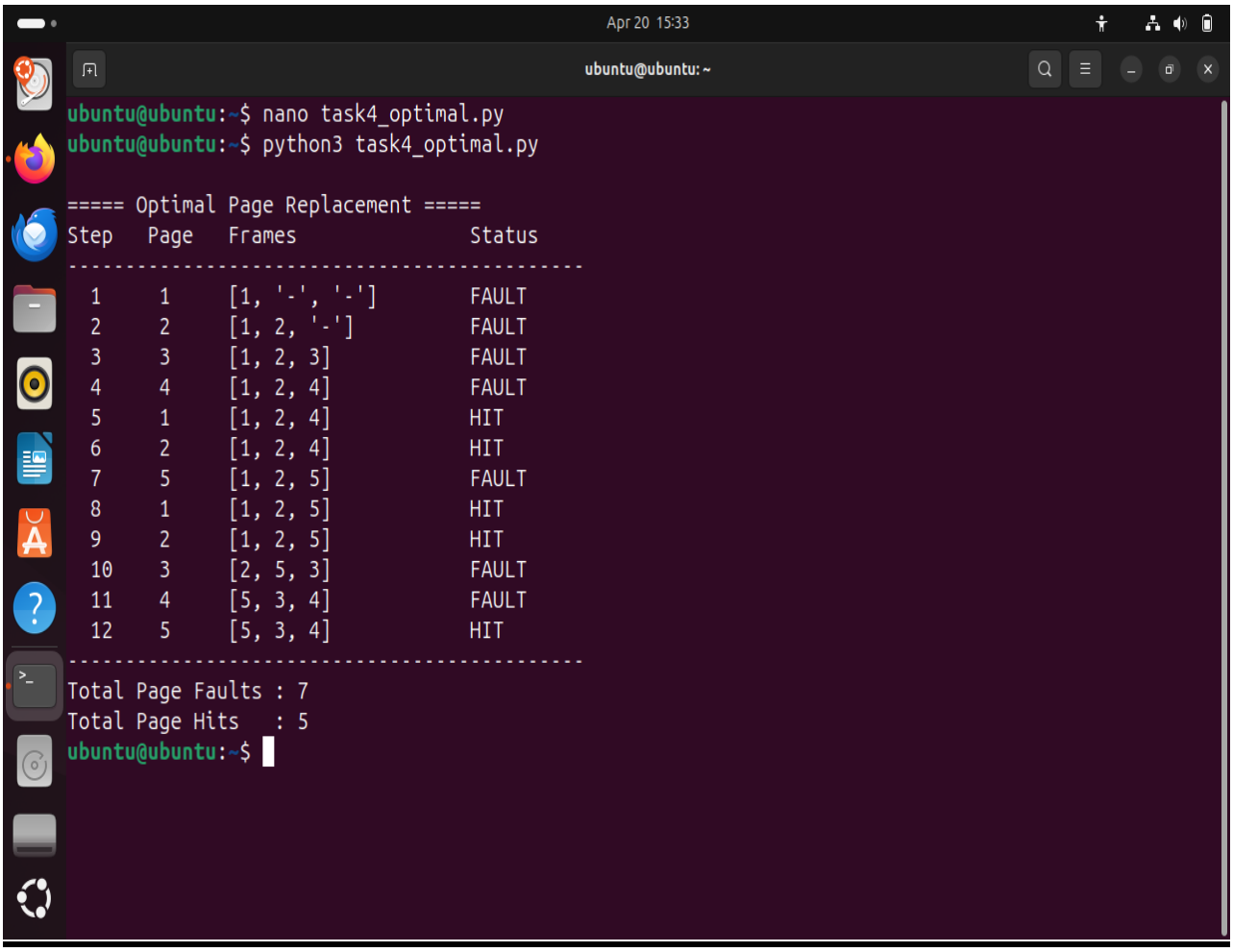


Task 4: Optimal Page Replacement

Code: -

```
pages = list(map(int, input("Enter pages: ").split()))
num_frames = int(input("Enter frames: "))

frames, faults = [], 0
print("\n===== OPTIMAL =====")
for i, page in enumerate(pages):
    fault = False
    if page not in frames:
        fault = True
        faults += 1
        if len(frames) < num_frames:
            frames.append(page)
        else:
            farthest, replace = -1, None
            for f in frames:
                nxt = next((j for j in range(i+1,len(pages)) if pages[j]==f), float('inf'))
                if nxt > farthest:
                    farthest, replace = nxt, f
            frames.remove(replace)
            frames.append(page)
    cur = frames[:] + ['-'] * (num_frames - len(frames))
    print(f"Step {i+1} | Page {page} | {cur} | {'FAULT' if fault else 'HIT'}")
print(f"Total Faults: {faults}")
```

Output:-



```
ubuntu@ubuntu:~$ nano task4_optimal.py
ubuntu@ubuntu:~$ python3 task4_optimal.py

===== Optimal Page Replacement =====
Step   Page   Frames              Status
-----------------------------------------
 1     1      [1, '-', '-']       FAULT
 2     2      [1, 2, '-']         FAULT
 3     3      [1, 2, 3]           FAULT
 4     4      [1, 2, 4]           FAULT
 5     1      [1, 2, 4]           HIT
 6     2      [1, 2, 4]           HIT
 7     5      [1, 2, 5]           FAULT
 8     1      [1, 2, 5]           HIT
 9     2      [1, 2, 5]           HIT
 10    3      [2, 5, 3]           FAULT
 11    4      [5, 3, 4]           FAULT
 12    5      [5, 3, 4]           HIT
-----------------------------------------
Total Page Faults : 7
Total Page Hits   : 5
ubuntu@ubuntu:~$
```

Task 5: MRU (Most Recently Used)

Code :-

```
pages = list(map(int, input("Enter pages: ").split()))
num_frames = int(input("Enter frames: "))

frames, faults = [], 0
print("\n===== MRU =====")
for i, page in enumerate(pages):
    fault = False
    if page not in frames:
        fault = True
        faults += 1
        if len(frames) < num_frames:
            frames.append(page)
        else:
            frames.remove(frames[-1])
            frames.append(page)
    else:
        frames.remove(page)
        frames.append(page)
    cur = frames[:] + ['-'] * (num_frames - len(frames))
    print(f"Step {i+1} | Page {page} | {cur} | {'FAULT' if fault else 'HIT'}")
print(f"Total Faults: {faults}")
```

Output:-

```
ubuntu@ubuntu:~$ nano task5_mru.py
ubuntu@ubuntu:~$ python3 task5_mru.py

===== MRU Page Replacement =====
Step   Page   Frames              Status
--------------------------------------------
 1     1     [1, '-', '-']       FAULT
 2     2     [1, 2, '-']         FAULT
 3     3     [1, 2, 3]           FAULT
 4     4     [1, 2, 4]           FAULT
 5     1     [2, 4, 1]           HIT
 6     2     [4, 1, 2]           HIT
 7     5     [4, 1, 5]           FAULT
 8     1     [4, 5, 1]           HIT
 9     2     [4, 5, 2]           FAULT
 10    3     [4, 5, 3]           FAULT
 11    4     [5, 3, 4]           HIT
 12    5     [3, 4, 5]           HIT
--------------------------------------------
Total Page Faults : 7
Total Page Hits   : 5
ubuntu@ubuntu:~$
```

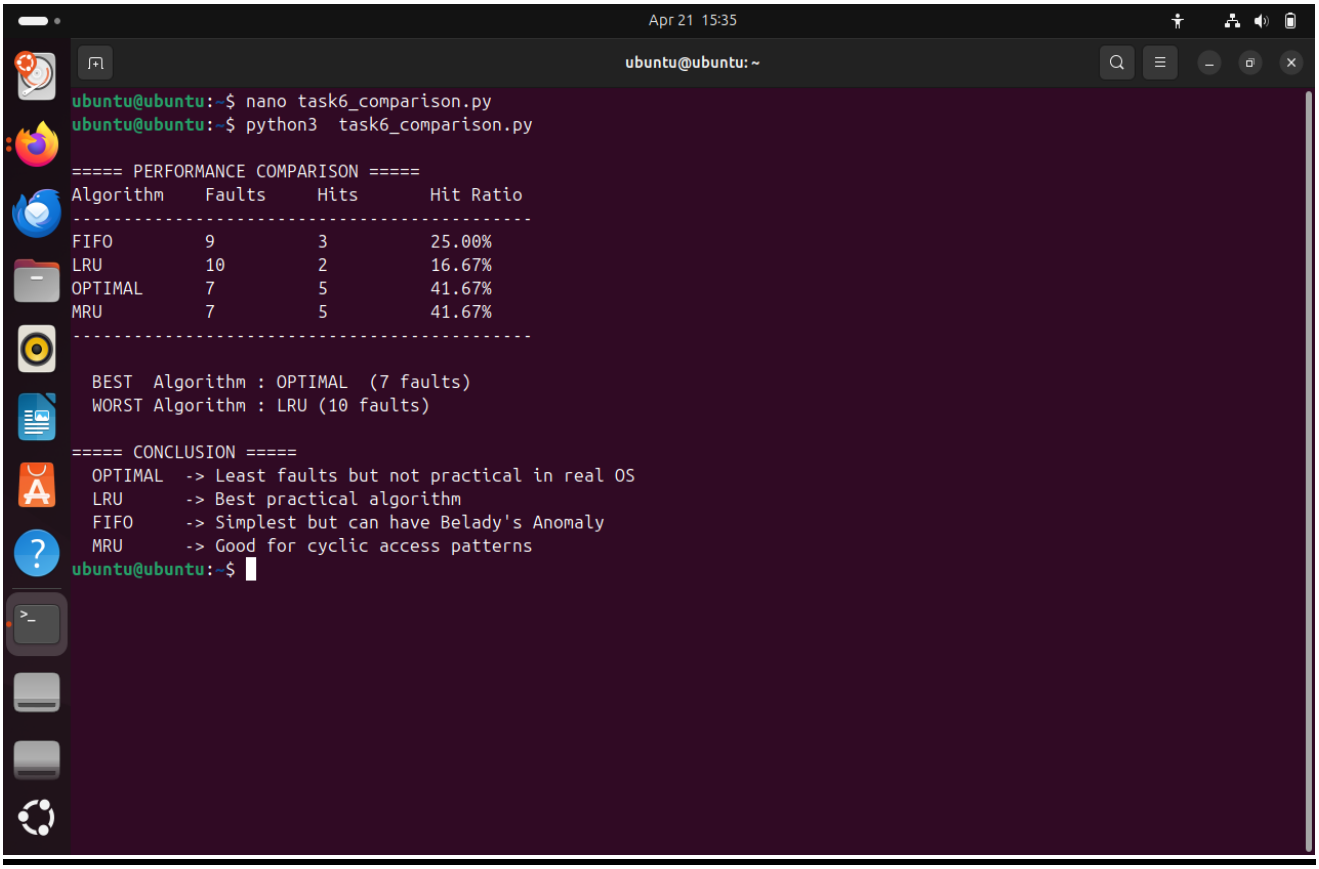
Task 6&7: Performance Comparison &Result

Code:-

```
from collections import deque
def fifo(pages, n):
    frames, frame_set, faults = deque(), set(), 0
    for i, page in enumerate(pages):
        if page not in frame_set:
            faults += 1
            if len(frames) < n:
                frames.append(page)
            else:
                frame_set.discard(frames.popleft())
                frames.append(page)
            frame_set.add(page)
    return faults

def lru(pages, n):
    frames, faults = [], 0
    for i, page in enumerate(pages):
        if page not in frames:
            faults += 1
            if len(frames) < n: frames.append(page)
            else:
                lru_page, lru_time = None, float('inf')
                for f in frames:
```

```
                last = next((j for j in range(i-1,-1,-1) if pages[j]==f), -1)
                if last < lru_time: lru_time, lru_page = last, f
            frames.remove(lru_page)
            frames.append(page)
        else:
            frames.remove(page)
            frames.append(page)
    return faults

def optimal(pages, n):
    frames, faults = [], 0
    for i, page in enumerate(pages):
        if page not in frames:
            faults += 1
            if len(frames) < n: frames.append(page)
            else:
                farthest, replace = -1, None
                for f in frames:
                    nxt = next((j for j in range(i+1,len(pages)) if pages[j]==f), float('inf'))
                    if nxt > farthest: farthest, replace = nxt, f
                frames.remove(replace)
                frames.append(page)
    return faults

def mru(pages, n):
```

```
    frames, faults = [], 0
    for i, page in enumerate(pages):
        if page not in frames:
            faults += 1
            if len(frames) < n: frames.append(page)
            else:
                frames.remove(frames[-1])
                frames.append(page)
        else:
            frames.remove(page)
            frames.append(page)
    return faults

pages = list(map(int, input("Enter pages: ").split()))
n = int(input("Enter frames: "))

results = {'FIFO': fifo(pages,n), 'LRU': lru(pages,n), 'OPTIMAL': optimal(pages,n),
'MRU': mru(pages,n)}

print("\n===== COMPARISON =====")
print(f"{'Algorithm':<12} {'Faults':<10} {'Hits':<10} {'Hit Ratio'}")
print("-" * 40)
for algo, faults in results.items():
    hits = len(pages) - faults
    print(f"{algo:<12} {faults:<10} {hits:<10} {(hits/len(pages))*100:.2f}%")
```

```
best  = min(results, key=results.get)
worst = max(results, key=results.get)
print(f"\nBEST  : {best} ({results[best]} faults)")
print(f"WORST : {worst} ({results[worst]} faults)")
print("\n===== CONCLUSION =====")
print("OPTIMAL  -> Minimum faults, not practical in real OS")
print("LRU      -> Best practical algorithm")
print("FIFO     -> Simple but Belady's Anomaly possible")
print("MRU      -> Good for cyclic access patterns")
```

Output:-